

BÁO CÁO KỸ THUẬT

THIẾT KẾ BÀN DI CHUỘT CẢM ỨNG TRÊN STM32F429I-DISCO

TouchGFX GUI • USB HID Mouse • FreeRTOS

Hạng mục	Thông tin
Vi điều khiển	STM32F429ZIT6, Cortex-M4
Kit	STM32F429I-DISCO
Màn hình	ILI9341, 240 × 320, RGB565
Cảm ứng	STMPE811
Giao diện	TouchGFX 4.26.1
Kết nối máy tính	USB HID Mouse, OTG_HS dùng Internal FS PHY

Tài liệu mô tả đúng phiên bản chương trình đã build và chạy trên kit.

Mục lục nội dung

- 1. Mục tiêu và kết quả đạt được
- 2. Kiến trúc phần cứng và phần mềm
- 3. Thiết lập clock, SDRAM và USB
- 4. Thiết lập USB HID Mouse
- 5. Thiết kế GUI TouchGFX
- 6. Logic điều khiển cảm ứng và chuột
- 7. Giải thích chi tiết các hàm
- 8. Luồng hoạt động tổng thể
- 9. Thông số hiệu chỉnh
- 10. Build, nạp và kiểm thử
- 11. Lỗi đã gặp và cách khắc phục
- 12. Kịch bản trình bày nhanh

1. Mục tiêu và kết quả đạt được

Đề tài xây dựng một bàn di chuột cảm ứng sử dụng trực tiếp màn hình cảm ứng của kit STM32F429I-DISCO. Chuyển động ngón tay được đổi thành độ dịch chuyển tương đối của chuột và gửi tới máy tính theo chuẩn USB HID, không cần cài driver riêng.

1.1 Chức năng hiện có

- Kéo ngón tay để di chuyển con trỏ theo độ dịch chuyển tương đối.
- Chạm ngắn rồi thả để thực hiện một lần click chuột trái.
- Nhấn giữ khoảng 0,5 giây rồi kéo để giữ chuột trái và drag-and-drop.
- Hiện thị vòng tròn tại vị trí chạm; vòng tròn đi theo ngón và co mọt khi thả.
- Tích lũy chuyển động trong khoảng -1024...1024 và chia thành nhiều HID report.
- Mỗi report giới hạn X/Y trong -127...127 theo định dạng int8_t.
- Kiểm tra USB configured và endpoint HID idle trước khi gửi để tránh ghi đè report.

Kết quả: Firmware đã build thành công thành tệp ELF/HEX và các chức năng GUI, di chuột, click trái, giữ-kéo đã được kiểm tra trên kit.

2. Kiến trúc phần cứng và phần mềm

Khối	Vai trò	Giao tiếp
STM32F429ZIT6	Xử lý TouchGFX, gesture và USB HID	Cortex-M4, 168 MHz
ILI9341	Hiện thị GUI 240×320	LTDC + SPI cấu hình
STMPE811	Đọc tọa độ chạm	I ² C3
SDRAM ngoài	Lưu double framebuffer và animation storage	FMC, 84 MHz
USB USER	Gửi HID report tới máy tính	OTG_HS Internal FS PHY
ST-LINK	Nạp/debug và cấp nguồn	SWD

2.1 Phân lớp chương trình

```

Touch input (STMPE811)
    ↓
TouchGFX ClickEvent / DragEvent / TickEvent
    ↓
Screen1View: nhận dạng tap, move, hold-drag, animation
    ↓
USB_Mouse_TrySend(buttons, dx, dy)
    ↓
USBD_HID_SendReport → USB OTG HS → máy tính
  
```

Tệp	Trách nhiệm
gui/src/screen1_screen/Screen1View.cpp	Logic cảm ứng, gesture, animation, hàng đợi chuyển động và phát report.
gui/include/gui/screen1_screen/Screen1View.h	Khai báo trạng thái và hàm của màn hình.
USB_DEVICE/App/usb_device.c	Khởi tạo USB Device HID và API gửi report an toàn.
Core/Src/main.c	Clock, SDRAM, LTDC, FreeRTOS, TouchGFX và khởi động USB.
Core/Inc/stm32f4xx_hal_conf.h	Thiết lập HAL tick priority.
STM32F429I_DISCO_REV_D01.ioc	Nguồn cấu hình CubeMX cho peripheral và middleware.

3. Thiết lập clock, SDRAM và USB

3.1 Clock hệ thống

Tham số	Giá trị	Ý nghĩa
HSE	8 MHz	Thạch anh ngoài trên kit.
PLLM	8	Đưa đầu vào PLL về 1 MHz.
PLLN	336	VCO đạt 336 MHz.
PLLQ	2	$SYSCLK = 336/2 = 168$ MHz.
PLLQ	7	$USB\ clock = 336/7 = 48$ MHz.
APB1	42 MHz	Peripheral clock miền APB1.
APB2	84 MHz	Peripheral clock miền APB2.
LCD-TFT	6 MHz	Pixel clock cho màn hình 240×320.

```

RCC_OscInitStruct.PLL.PLLM = 8;
RCC_OscInitStruct.PLL.PLLN = 336;
RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
RCC_OscInitStruct.PLL.PLLQ = 7;
  
```

3.2 SDRAM và framebuffer

FMC chạy với SDRAM clock 84 MHz. Refresh count được đặt 1292. TouchGFX sử dụng double framebuffer RGB565 trong SDRAM; mỗi framebuffer cần $240 \times 320 \times 2 = 153.600$ byte. Ngoài ra còn có animation storage. Linker đặt vùng TouchGFX_Framebuffer tại 0xD0000000.

```
#define REFRESH_COUNT ((uint32_t)1292) /* SDRAM refresh at 84 MHz */
```

3.3 HAL tick

TIM6 làm timebase 1 ms cho HAL. Priority phải giữ ở 0 để HAL_Delay vẫn chạy trong giai đoạn TouchGFX/FreeRTOS tạo critical section. Nếu priority là 15, chương trình có thể kẹt khi STMPE811 reset và màn hình chỉ trắng.

```
#define TICK_INT_PRIORITY 0U
```

4. Thiết lập USB HID Mouse

4.1 Lựa chọn peripheral

- Connectivity → USB_OTG_HS.
- Internal FS PHY → Device_Only.
- External PHY → Disable.
- PB14 = USB_OTG_HS_DM; PB15 = USB_OTG_HS_DP.
- Middleware USB_DEVICE → HID cho HS IP.
- USB clock bắt buộc đúng 48 MHz.

Lý do không dùng USB_OTG_FS: PA11/PA12 đang được LTDC sử dụng cho hai bit màu đỏ R4/R5. OTG_HS ở Internal FS PHY dùng PB14/PB15 nên giữ nguyên giao diện LCD.

4.2 Cấu trúc HID report

Byte	Trường	Kiểu/miền
0	Buttons	bit 0 = nút trái
1	Delta X	int8_t, -127...127
2	Delta Y	int8_t, -127...127
3	Wheel	0 trong phiên bản hiện tại

```
report[0] = buttons;
report[1] = (uint8_t)deltaX;
report[2] = (uint8_t)deltaY;
report[3] = 0;
```

4.3 API USB_Mouse_TrySend()

Hàm là lớp cầu nối C giữa GUI C++ và USB middleware C. Hàm trả về 1 khi report thực sự được nhận để truyền; trả 0 nếu USB chưa configured hoặc endpoint còn busy.

```
uint8_t USB_Mouse_TrySend(uint8_t buttons,
                          int8_t deltaX,
                          int8_t deltaY);
```

1. Kiểm tra `hUsbDeviceHS.dev_state == USB_STATE_CONFIGURED`.
2. Lấy `USB_HID_HandleTypeDef` từ `pClassData`.
3. Chỉ gửi khi `hid->state == USB_HID_IDLE`.
4. Gọi `USB_HID_SendReport()` với report 4 byte.

5. Thiết kế GUI TouchGFX

5.1 Cấu trúc Screen1

Widget	Vai trò
box1	Nền toàn màn hình 240×320.
circle1	Vòng tròn tương tác chính; bán kính thay đổi 40→0.
circle2–circle4	Widget cũ vẫn tồn tại trong Designer nhưng được ẩn; logic tối ưu chỉ dùng circle1.
Canvas buffer	3600 byte cho Canvas Widget Renderer.

5.2 Quy tắc invalidate

Trước khi ẩn hoặc di chuyển Circle, vùng cũ phải được invalidate trong lúc widget còn visible. Sau khi đổi vị trí/bán kính, invalidate vùng mới. Thứ tự này buộc TouchGFX vẽ lại nền và ngăn các cung tròn trắng lưu vết trên màn hình.

```
circle1.invalidate(); // vùng cũ
circle1.setXY(newX, newY);
circle1.invalidate(); // vùng mới
```

5.3 Animation

Khi thả tay, `animationRadius` bắt đầu ở 40 và giảm 2 pixel mỗi tick. Với khoảng 60 FPS, hiệu ứng kéo dài khoảng 20 frame, tương đương 0,33 giây. Chỉ một Circle được sử dụng nên giảm số vùng redraw và tải Canvas Renderer.

6. Logic điều khiển cảm ứng và chuột

Gesture	Điều kiện	Hành động
Di chuột	Có <code>DragEvent</code> khi đang pressed	Tích lũy delta X/Y và gửi report buttons=0.
Click trái	3–18 tick và di chuyển ≤4 px	Gửi buttons=1, tick sau gửi buttons=0.
Giữ-kéo	Giữ ≥30 tick, di chuyển ban đầu ≤4 px	Đặt buttons=1; các delta tiếp theo giữ nút.
Nhả drag	RELEASED khi dragButtonDown	Đặt requestedButtons=0.
Animation	Sau RELEASED	Bán kính giảm 2 mỗi tick cho tới 0.

6.1 Vì sao click không gửi ngay khi PRESSED?

Nếu gửi nút trái ngay khi vừa chạm, mọi thao tác rê con trỏ sẽ bị hiểu thành kéo đối tượng. Chương trình chờ tới RELEASED để phân loại tap. Với hold-drag, nút trái chỉ được bật sau ngưỡng giữ. Đây là cách tách ba gesture move, tap và drag.

6.2 Hàng đợi chuyển động

DragEvent có thể đến nhanh hơn endpoint HID. pendingDeltaX/Y giữ phần chưa gửi, giới hạn trong -1024...1024. Mỗi lần endpoint rảnh, clampMouseDelta tách tối đa 127 đơn vị. Sau khi gửi thành công, phần đã gửi được trừ khỏi hàng đợi; phần còn lại gửi ở tick sau.

```
pendingDeltaX/Y: -1024 ... 1024
reportX/Y:       -127 ... 127
pendingDeltaX -= reportX;
pendingDeltaY -= reportY;
```

7. Giải thích chi tiết các hàm

Screen1View::Screen1View()

Khởi tạo toàn bộ state: pressed, releaseAnimation, trạng thái button, bộ đếm tick, tọa độ và hàng đợi delta về giá trị an toàn.

setupScreen()

Gọi setup của lớp generated rồi đảm bảo tất cả Circle ban đầu bị ẩn.

tearDownScreen()

Chuyển tiếp thao tác kết thúc màn hình về Screen1ViewBase.

hideAllCircles()

Duyệt 4 Circle; chỉ invalidate và ẩn widget đang visible để giảm redraw và không lưu vết.

handleClickEvent()

Xử lý PRESSED/RELEASED; khởi tạo trạng thái gesture, xác định click/nhả drag và bắt đầu animation.

handleDragEvent()

Tính delta từ oldX/Y tới newX/Y, cộng movementDistance, queue chuyển động và cập nhật vị trí Circle.

handleTickEvent()

Tăng pressTicks, kích hoạt hold-drag, phục vụ USB queue và cập nhật animation mỗi frame.

queueMouseMovement()

Nhân delta với POINTER_GAIN=1, cộng vào hàng đợi và bảo hòa trong -1024...1024.

serviceUsbMouse()

Nếu có delta/button mới thì clamp report, thử gửi USB; chỉ cập nhật state khi gửi thành công.

clampMouseDelta()

Ép một thành phần delta về miền hợp lệ của int8_t: -127...127.

USB_Mouse_TrySend()

Kiểm tra USB và HID endpoint rồi tạo/gửi report 4 byte.

MX_USB_DEVICE_Init()

Khởi tạo USB core, đăng ký HID class và start USB device.

StartDefaultTask()

Khởi tạo USB_DEVICE sau khi scheduler bắt đầu, sau đó chạy vòng lặp task nền.

8. Luồng hoạt động tổng thể

8.1 Khởi động

5. HAL_Init và cấu hình TIM6 timebase.
6. SystemClock_Config: SYSCLK 168 MHz, USB 48 MHz.
7. Khởi tạo GPIO, CRC, I²C3, SPI5, FMC/SDRAM, LTDC, DMA2D.
8. MX_TouchGFX_Init và khởi tạo STMPE811.
9. Khởi tạo FreeRTOS; tạo defaultTask và GUI_Task.
10. defaultTask gọi MX_USB_DEVICE_Init.
11. GUI_Task chạy vòng lặp TouchGFX và nhận sự kiện cảm ứng.

8.2 Tap-click

```

PRESSED → reset pressTicks/movementDistance
↓
Tick tăng thời gian giữ
↓
RELEASED → kiểm tra  $3 \leq \text{ticks} \leq 18$  và  $\text{distance} \leq 4$ 
↓
requestedButtons=1 → gửi report press
↓ tick kế tiếp
requestedButtons=0 → gửi report release

```

8.3 Hold-drag

```

PRESSED → giữ  $\geq 30$  tick
↓
requestedButtons=1
↓
DragEvent → report(button=1, dx, dy)
↓
RELEASED → report(button=0, 0, 0)

```

9. Thông số hiệu chỉnh

Hàng số	Giá trị	Tác động
HOLD_TICKS	30	Khoảng 0,5 s trước khi vào chế độ giữ-kéo.
MIN_TAP_TICKS	3	Bỏ tiếp xúc nhiều quá ngắn, khoảng dưới 50 ms.
MAX_TAP_TICKS	18	Tap dài hơn khoảng 300 ms không bị tính click.
TAP_MOVEMENT_LIMIT	4 px	Ngăn click nhầm khi ngón đã rê.
POINTER_GAIN	1	Độ nhạy con trỏ 1:1 với delta cảm ứng.
ANIMATION_START_RADIUS	40 px	Bán kính ban đầu.

ANIMATION_RADIUS_STEP	2 px/tick	Tốc độ co; khoảng 20 frame.
Pending delta limit	±1024	Chống tràn khi USB tạm bận/ngắt kết nối.
HID delta limit	±127	Giới hạn trường X/Y của report.

Điều chỉnh nhanh: Muốn chuột chậm hơn có thể xử lý theo tỉ lệ phân số hoặc bỏ delta nhỏ; muốn khó click nhằm hơn thì giảm TAP_MOVEMENT_LIMIT hoặc tăng MIN_TAP_TICKS.

10. Build, nạp và kiểm thử

10.1 Build

- Mở project STM32F429I_DISCO_REV_D01 trong STM32CubeIDE.
- Chọn cấu hình Debug và nhấn Build.
- Kết quả chính: STM32CubeIDE/Debug/STM32F429I_DISCO_REV_D01.elf và .hex.

10.2 Kết nối cáp

- Cáp ST-LINK phía trên: nạp/debug và cấp nguồn.
- Cáp USB USER phía dưới màn hình: truyền USB HID tới máy tính.
- Cáp USB USER phải hỗ trợ data, không dùng cáp chỉ sạc.

10.3 Checklist kiểm thử

Kiểm thử	Kết quả mong đợi
Khởi động	GUI xuất hiện, không kẹt màn hình trắng.
Chạm	Circle xuất hiện đúng tọa độ.
Rê	Con trỏ di chuyển đúng hướng, không click.
Tap	Một lần click trái, không kẹt nút.
Giữ-kéo	Đối tượng trên PC kéo theo con trỏ và được thả khi nhấc tay.
Animation	Circle co mướt và nền được phục hồi, không lưu vết.
USB reconnect	Không ghi đè report khi endpoint busy; hoạt động lại khi configured.

11. Lỗi đã gặp và cách khắc phục

Hiện tượng	Nguyên nhân	Khắc phục
Không biên dịch GUI	Biến khai báo không khớp và handleDragEvent bị định nghĩa hai lần.	Đồng bộ .hpp/.cpp và bỏ hàm trùng.
Màn hình trắng	TIM6 priority=15 làm HAL tick bị che trong critical section.	Đặt TICK_INT_PRIORITY và TIM6 về 0.
USB không nhận	Chưa có USB HID middleware hoặc	Bật USB_DEVICE HID và dùng cáp data

	chưa cắm cổng USB USER.	ở cổng USER.
Clock USB sai 90 MHz	PLL 180 MHz với PLLQ=4.	Đổi PLLN=336, PLLQ=7, SYSCLK=168 MHz.
Vòng trắng lưu vết	Ẩn/di chuyển widget trước khi invalidate vùng cũ.	Invalidate vùng cũ khi Circle còn visible.
Vòng nhỏ không xuất hiện	Widget bị bật và tắt trong cùng tick.	Tách các bước animation theo tick.
Click nhầm/chuột quá nhạy	Gain=2 và ngưỡng tap rộng.	Gain=1; tap 3–18 tick và ≤4 px.

12. Kịch bản trình bày nhanh

12.1 Bài nói 3–5 phút

15. Mục tiêu: biến màn hình cảm ứng của STM32F429I-DISCO thành touchpad USB không cần driver.
16. Phần cứng: STM32F429, STMPE811, ILI9341, SDRAM, USB USER.
17. TouchGFX tạo sự kiện Press/Drag/Release; Screen1View phân loại move, tap và hold-drag.
18. Chuyển động được tích lũy và chia thành report 4 byte; chỉ gửi khi USB configured và HID idle.
19. GUI dùng một Canvas Circle, invalidate đúng vùng và co bán kính theo từng frame.
20. Các lỗi quan trọng đã xử lý: clock USB 48 MHz, TIM6 priority, lưu vết Canvas và click nhầm.
21. Kết quả: di chuột, click trái, drag-and-drop và animation đều hoạt động trên phần cứng.

12.2 Demo đề xuất

22. Mở Notepad hoặc desktop để thấy con trỏ.
23. Rê ngón tay chậm: chứng minh chuyển động tương đối.
24. Tap vào biểu tượng/nút: chứng minh click trái.
25. Giữ rồi kéo một cửa sổ hoặc biểu tượng: chứng minh drag-and-drop.
26. Nhắc tay và chỉ vào hiệu ứng Circle co mở rộng.

12.3 Câu hỏi thường gặp

Câu hỏi	Trả lời ngắn
Vì sao dùng HID?	Hệ điều hành có driver chuột HID sẵn, không cần cài phần mềm.
Vì sao OTG_HS nhưng chỉ Full Speed?	Dùng core HS với Internal FS PHY trên PB14/PB15 để tránh PA11/PA12 của LCD.
Vì sao không gửi click ngay khi chạm?	Để phân biệt thao tác rê con trỏ với click và kéo-thả.
Vì sao cần hàng đợi delta?	Tần số sự kiện cảm ứng có thể cao hơn tốc độ endpoint HID.
Vì sao SYSCLK giảm từ 180 xuống 168 MHz?	Cần tạo chính xác USB clock 48 MHz bằng PLLQ=7.
Có thể mở rộng gì?	Click phải, scroll, double-click, gesture hai ngón, UI chỉnh sensitivity.

Kết luận

Hệ thống đã tích hợp thành công ba lớp chính: cảm ứng và GUI TouchGFX, thuật toán nhận dạng gesture, và USB HID Mouse. Thiết kế hiện tại ưu tiên độ ổn định: không ghi đè report đang truyền, không click ngay khi vừa chạm, có giới hạn hàng đợi và có cơ chế phục hồi vùng vẽ. Đây là nền tảng phù hợp để mở rộng thêm click phải, cuộn, đa điểm chạm và màn hình cấu hình.